TECHNICAL BRIEF

MeshTab: Cross-Currency Group Settlement on Monad

Close the tab. Keep the friends.

October 2026 | Version 1.0

Document Information

Product Name	MeshTab
Team	**Osasumwen Ngozi Scott**: Co-founder and Product Lead (ONSInsights) **Miracle Aigbomian**: Co-founder and Developer (miracles2motion)
Scope	Architecture and implementation of the testnet prototype, the evidence behind its claims, its known limits, and a corrected mainnet roadmap
Network	Monad Testnet (chain ID 10143)
Document Status	Final
Basis	Facts come from the developer's code reading and test reports of October 12, 2026, and from the public on-chain addresses in Section 3. Anything that could not be confirmed is marked as such.
Companion Documents	PRD v1.0, Product Brief v1.0, GTM Strategy v1.0
Last Updated	October 2026

1. SUMMARY

MeshTab is a hybrid system. An application server holds tabs, expenses, balances and the netting calculation. A small ledger contract on Monad executes the token transfers and emits an event for each. Users sign in with a passkey, a smart account is created for them automatically, and a paymaster pays testnet gas. This brief describes what is built, how it behaves, what has been measured, and what is not yet true.

Layer	Choice
Front end	Next.js 16 and React 19, delivered as a web app with offline drafting
Identity	Privy: passkeys, Google or email, with embedded smart wallets
Accounts	ERC-4337 LightAccount smart accounts
Gas	Pimlico bundler and verifying paymaster, sponsored on testnet
Chain	Monad Testnet. Published figures: 400 ms blocks, 800 ms finality
Settlement token	The issuer's official Agora testnet AUSD, 6 decimals
Ledger contract	MeshTabAgoraLedger: permissionless, one payer-to-recipient transfer per call, unaudited
Data	Upstash Redis with Lua scripts for atomic updates, in integer cents
FX	Pyth Hermes: six live feeds, three indicative currencies
Receipt scan	Google Gemini Flash through a server proxy

2. ARCHITECTURE

2.1 Layers

Layer	Components	Responsibility
Client	Next.js app, netting canvas, offline outbox, settlement drawer	Capture expenses, compute and display netting, build and send the settlement operation
API	Next.js API routes with Privy bearer-token checks	Authenticate, check tab membership, run atomic updates, proxy OCR and FX
Data	Upstash Redis and Lua scripts	Tabs, members, expenses, balances, activity feed, join codes
Identity	Privy	Passkey sign-in, embedded wallets, session signing, dashboard sponsorship policies
Account abstraction	LightAccount, Pimlico bundler and paymaster	Batch calls into one operation and sponsor its gas
Chain	Monad Testnet: ledger, Agora AUSD, Agora faucet	Token transfers, receipts and events
External services	Pyth Hermes, Google Gemini	Exchange rates and receipt reading

2.2 Four Main Flows

- **Log an expense.** The client writes to a local outbox, then pushes to the API. The API checks the Privy token and tab membership, then a Lua script updates the expense and every member's balance atomically in Redis. Other members see it on their next poll.
- **Net.** The client reads the tab, computes each member's net position and the greedy payment list, and compares it with a per-currency baseline. Netting is off-chain and is never sent to the contract.
- **Settle.** The client builds one batch: an approval for the total, then one payment call per creditor. The smart account sends it as one operation through the bundler, with gas sponsored. The device first asks for confirmation. The ledger emits a DebtSettled event per payment and the app shows the receipt with an explorer link.
- **Fund (testnet).** Add Funds sends a sponsored call to the Agora faucet, which transfers 10,000 testnet AUSD to the wallet.

3. ON-CHAIN COMPONENTS

Contract	Address (Monad Testnet)	Purpose
MeshTabAgoraLedger	0x54f02333ea8D80C175A1C8Bdb71ef709cF424C83	Executes token transfers from the caller to a creditor and emits an event
Agora AUSD (testnet)	0xa9012a055bd4e0eDfF8Ce09f960291C09D5322dC	The issuer's official testnet token, 6 decimals
Agora faucet	0xd236c18D274E54FAccC3dd9DDA4b27965a73ee6C	Issues test tokens: 10,000 per claim, 100,000 wallet ceiling, one claim per 60 seconds

3.1 Ledger Behavior

- Constructor takes the token address (`_ausdToken`). The token is fixed at deployment, so using another token means a new deployment.
- `payNettedDebt(string groupId, address creditor, uint256 amount)` transfers tokens from the caller's own approved balance directly to the creditor. It is permissionless.
- Each call emits `DebtSettled(string groupId, address indexed debtor, address indexed creditor, uint256 amount)`. A read function `ausdToken()` returns the token.
- The contract holds no funds, has no owner, pause or limit functions in the reported interface, and does not verify

netting or tab membership. Netting is off-chain.

- No member can move another member's funds, which is why a group cannot settle in a single transaction.
- `groupId` is a plain text identifier (for example `lisbon-24` or a tab code) and is emitted as is, not hashed.
- Hardhat is configured to verify the source on Monadscan and Sourcify. Whether verification has been completed was not confirmed, so the contract is described as deployed.
- The contract is unaudited.

4. ACCOUNTS, GAS AND SETTLEMENT

4.1 Smart Accounts and Sponsorship

- Each user gets a Privy embedded passkey wallet, which is an ERC-4337 LightAccount.
- Pimlico's bundler and verifying paymaster are configured through the Privy dashboard for Monad Testnet.
- Sponsorship covers only three whitelisted contracts: the ledger, the Agora AUSD token and the Agora faucet. Any other target is rejected.
- A monthly testnet spend ceiling with an alert at 80% protects the budget. There is no per-user gas cap on testnet.
- Users pay no gas for faucet claims, approvals or settlements.

4.2 The Settlement Operation

When the debtor taps Confirm Settlement, the client sends one batch call: an `approve` for the total on the token, then one `payNettedDebt` per creditor. The smart account runs it through `executeBatch`, and the bundler submits it as one operation in one block. For a debtor who owes two people (the "You" member in the Lisbon example), the developer's code reading is that one operation covers both creditors. Only a single-payment settlement was timed.

4.3 Confirmation on the Device

Device state	What the user sees before the payment is sent
Passkey enrolled	The operating system's biometric prompt (Face ID, Touch ID or Windows Hello). If approved the operation is sent. If cancelled nothing is sent.
No passkey, consumer view	A prompt offering to enroll a passkey. If the user continues without one, a confirmation dialog (slide to confirm or type the amount) appears and the Privy session key signs without a biometric prompt.
Developer or sample mode	The payment is confirmed and signed immediately, with no biometric prompt.

A private-key export for the embedded wallet exists only in Developer Mode, behind a biometric gate. Consumer screens never mention or require a seed phrase.

4.4 Measured Speed

Tap-to-confirmed time was measured by stopwatch on the single-payment batched flow, 10 runs on two devices (an iPhone 15 Pro in Safari and a Mac M2 in Chrome) over Wi-Fi, 5G and a cellular hotspot. The full run table is in Appendix C.

Measure	Result
Median	1,275 ms (about 1.3 seconds)
Fastest	1,190 ms
Slowest	1,540 ms (cellular hotspot)
Before batching (two sequential operations)	Median 2,790 ms, slowest 3,480 ms
Protocol reference (published)	400 ms blocks, 800 ms finality

It was not stated whether the biometric prompt time is included, and multi-creditor settlements were not timed. An earlier RPC

round-trip benchmark measured the network, not a settlement, and is supporting data only.

5. NETTING AND MONEY MODEL

5.1 Method

- **One dollar unit.** Each expense is converted to dollars once, at entry, and stored in whole cents. Six currencies (EUR, GBP, JPY, CAD, AUD, CHF) use live Pyth Hermes rates through a server route. SGD, AED and NGN use labeled indicative rates. The sample Lisbon trip uses fixed seed rates (EUR 1.09, GBP 1.28) so the demo is repeatable.
- **Integer cents everywhere.** Conversion to the token's 6 decimals happens only at the contract boundary: units = cents x 10,000.
- **Deterministic rounding.** Equal splits that do not divide evenly hand out the extra cents in a fixed order. Balances always sum to $0.00.
- **Net positions.** Paid minus fair share, per member.
- **Greedy matching.** Debtors and creditors are paired in roster order until every position is zero. This gives at most (members with a non-zero balance minus 1) payments. It does not sort by size and is not guaranteed to be the theoretical minimum.

5.2 Per-Currency Baseline

To show the benefit, the app runs the same greedy netting inside each currency separately and adds up the payments (`computePerCurrencyBaseline`). For the Lisbon sample this gives 9 payments against 3 after cross-currency netting, shown as "9 transfers to 3 direct" and "6 fewer payments." A single-currency tab shows no reduction, because the two numbers match. The developer's per-currency breakdown (4, 2 and 3) differs from a hand calculation (3, 3 and 3), but both sum to 9, so this brief uses only the total.

5.3 Property Tests

Script: `test-netting-properties.mjs`. 1,000 randomized tabs with 3 to 20 members, 3 to 17 expenses, uneven splits, and expenses in five currencies (USD, EUR, GBP, JPY, CAD).

Invariant	Meaning	Status
Conservation of value	Sum of balances is exactly $0.00	Tested: 1,000 of 1,000 passed
Bounded edge count	Payments never exceed (non-zero members minus 1)	Tested: 1,000 of 1,000 passed
Exact liquidity	Payments total the sum of positive balances	Algorithmic property, not covered by the reported run
Termination and positivity	Finite steps, no loops, every payment above zero	Algorithmic property, not covered by the reported run

The PRD's long-term target is 10,000 tabs. That extension is a trial-count change and is planned before any mainnet use.

5.4 FX Exposure

Obligations are fixed in dollars at the rate on the day each expense was entered. A member whose home currency moves before they cash out bears that difference. MeshTab does not hedge it, and cash-out to local currency is not built.

6. DATA LAYER AND SYNC

6.1 Redis

Keys are partitioned by mode: `meshtab:${mode}:tab:${tabId}`, with an activity feed under `${tabKey}:activity`. Tab data keeps a 30-day expiry that refreshes when expenses are added. Join codes

have 8 characters.

Lua script	What it does
LUA_ADD_EXPENSE	Checks the expense does not already exist, adds it in timestamp order, updates each member's balance, refreshes the 30-day expiry
LUA_EDIT_EXPENSE	Reverses the earlier balance effect and applies the new one, updating the expense record
LUA_DELETE_EXPENSE	Reverses the balance effect, flags the expense as deleted and removes it from the ordered list
LUA_REMOVE_MEMBER	Rejects removal if the balance is not zero. Otherwise removes the member

Balances are stored in integer cents. The scripts apply them with HINCRBYFLOAT. Because the values are whole numbers this is exact, but moving to HINCRBY is a mainnet-readiness item.

6.2 What Lives Where

Off-chain (app server and Redis)	On-chain (Monad Testnet)
• Tabs, members, join codes • Expenses, balances and history • The netting calculation • Activity feed and nudges • Sync state	• The token transfer for each payment • A DebtSettled event with the tab identifier, payer, recipient and amount • Faucet claims

6.3 Sync and Offline

- Expense changes go to a local outbox (localStorage). Web Locks stop two browser tabs from flushing the same queue.
- The outbox pushes with a 500 ms debounce. A background retry loop runs every 60 seconds. The active tab is polled every 5 seconds when online.
- Conflicts resolve last-write-wins on server arrival, and the change is attributed.
- Sample-mode storage keys use the `meshTab_sample_` prefix with migrate-on-read fallbacks, so clearing sample data never touches testnet data.
- Settlement needs a connection. There is no offline settlement.

7. API SURFACE

Endpoint	Method	Access	Notes
/api/health	GET	Public	Status check
/api/pyth	GET	Public	FX rates through a server route
/api/receipt-scan	POST	No auth listed	Gemini vision proxy. Rate limited to 10 requests a minute, in memory
/api/tab/[id]/expenses	GET	Privy token and membership	List expenses
/api/tab/[id]/expense/new	POST	Privy token and membership	Add an expense (atomic)
/api/tab/[id]/expense/[id]	PATCH, DELETE	Privy token and membership	Edit or soft-delete
/api/tab/[id]/settle	POST	Privy token and membership	Record a settlement
/api/tab/[id]/members	POST	Privy token and membership	Manage members
/api/tab/[id]/remind	POST	Privy token and membership	In-app nudge (Ping All)
/api/tab/[id]	DELETE	Privy token and membership	Delete a tab
/api/tab/new	POST	Privy token	Create a tab

Endpoint	Method	Access	Notes
/api/tab/join	POST	Privy token	Join by link or code. Enforces the 20-member cap
/api/tab/user	GET	Privy token	The user's tabs
/api/user/profile	POST	Privy token	Display name and avatar

8. MODES AND PROTOTYPE SURFACES

Setting	Options	Where
Workspace mode	Sample (default for judges, Lisbon trip pre-filled), blank (clean sandbox), testnet (real transactions)	Settings, Workspace Sandbox Mode, or the landing buttons (Launch Lisbon Demo, Connect Wallet)
View	Consumer (default, no wallet jargon) or Developer and Web3 (raw wallet modals, telemetry, private-key export)	Settings, Security tab, Developer and Web3 toggle

Prototype surfaces, not live. The Cash Out and Virtual Card screens are simulations. On testnet, Cash Out says that off-ramping to a bank is simulated. They collect no bank or routing numbers: the user picks a destination type. **In-app nudge.** Ping All posts a notification and a payment-request item to the tab's activity feed. It sends no email, SMS or push.

9. SECURITY AND PRIVACY

Control	Status
Authentication	Privy bearer token on tab and user endpoints, plus a tab-membership check on tab endpoints
Secrets	Server-only keys (Gemini, Redis token) are not exposed to the client. Variables with the public prefix are non-secret
Contract risk	Permissionless and caller-pays. Holds no funds. No owner functions in the reported interface. Unaudited
Gas abuse	Whitelist of three contracts, monthly ceiling with alert, faucet cooldown and ceiling on-chain. No per-user cap
OCR abuse	10 requests a minute per IP, in memory (per server instance, so best effort)
Capacity	Hard cap of 20 members, enforced in the join endpoint and the UI
On-chain privacy	Addresses, amounts and a plain text tab identifier. Names, descriptions and receipts are never on-chain
Custody	None. Wallets are user-controlled smart accounts

10. KNOWN LIMITATIONS

Limitation	Why it matters	Plan
The ledger does not verify netting or membership	The calculation is trusted off-chain	Property tests now. Consider on-chain commitments later
The tab identifier is plain text on-chain	If it matches a join code, anyone reading the explorer could use it	Use an opaque or hashed identifier before mainnet
HINCRBYFLOAT on integer-cent values	Exact for whole numbers, but a floating-point API	Move to HINCRBY
In-memory rate limit	Per server instance, weak on serverless hosting	Durable limiter

Limitation	Why it matters	Plan
No per-user sponsorship cap	A bot could exhaust the sponsored budget	Monthly ceiling and alert now. Per-user caps later
Signing without a biometric prompt on some paths	No-passkey devices and developer or sample modes	Require re-authentication before real money
Off-chain state is the source of truth	History could change without an on-chain trace	Change attribution now. Commitments later
Last-write-wins sync	Concurrent offline edits can overwrite each other	Attribution shown. Revisit if needed
Three indicative currencies and fixed sample rates	Not live FX	Labeled in the UI
Multi-creditor settlement not timed	The 1.3 second figure is for a single payment	Time it after submission
Two copy lines to fix after the freeze	The capacity message says instant, and a Settings line says the account is secured by the biometric enclave, which is not true on the no-passkey path	Reword
The first proof settlement was run by a script	It predates the batched flow	Live-app hashes provided (Appendix B)

11. MAINNET PATH (ROADMAP)

This section corrects and condenses the developer's draft production architecture. Nothing here is live. Each item is a plan, and several depend on partner coverage that is not confirmed.

11.1 Principles

- **Non-custodial.** Users keep control of their accounts.
- **Zero-collateral participation.** Nobody deposits to join a tab. The cost is that a debtor can default, because nothing forces payment.
- **Just-in-time settlement.** Money moves only when a debtor settles.
- **Off-chain netting, on-chain transfer.** As today.

11.2 Target Architecture

Layer	Today (testnet)	Mainnet target
Identity and accounts	Passkey smart accounts through Privy	Same, with re-authentication for transfers
Netting	Off-chain, greedy, up to 20 members	Same. Revisit the cap and algorithm with measured latency
Settlement	Per-call ledger, one batched operation, official testnet AUSD	Production token with a ledger redeployed against it. Consider an opaque tab identifier and on-chain commitments of net positions
Gas	Sponsored by a testnet credit, no per-user cap	Funded paymaster, per-user caps, monitoring
Funding and cash-out	Simulated	Through Privy's funding features (Stripe on-ramp) and bank cash-out, subject to network, asset and currency coverage. NGN is unconfirmed
Card	Simulated	Through Privy's card program, subject to program access and supported assets
Reminders	In-app Ping All	Push, email and SMS
Budget guardrail	Not built	Concept: an advisory view of balance against open tab liabilities. A non-custodial wallet cannot be blocked from withdrawing, and any hold would need a custody design and legal review

11.3 Prerequisites Before Mainnet

- An independent security audit, plus the ack3 security scan if the team is selected for that sponsor prize.
- A legal opinion on fees, funding, cards and any balance-based feature.
- A redeployed ledger against the production token, a funded paymaster, per-user caps and durable rate limiting.
- HINCRBY for balances, an opaque tab identifier, and re-authentication for transfers.
- Property tests extended to 10,000 tabs, plus a timing run for multi-creditor settlements and a load test at the 20-member cap.
- Monitoring and alerts for the paymaster, faucet-equivalent flows and the API.

11.4 Worked Example (Corrected)

A pays $600 for a rental split three ways ($200 each for A, B and C). B pays $300 for a car split between A and B ($150 each). Raw debts: B owes A $200, C owes A $200, and A owes B $150. Net positions: **A +$250, B −$50, C −$200**. Netting gives two payments: B pays A $50 and C pays A $200. Three raw transfers become two.

11.5 What We Do Not Claim

Netting does not remove a fixed percentage of payments. It does not scale to hundreds of participants, because tabs are capped at 20 and netting runs off-chain. The settlement contract is not formally verified. Fiat funding, card and cash-out are not live. Nothing enforces repayment.

12. CONFIGURATION

Variable (name only)	Kind
UPSTASH_REDIS_REST_URL	Server secret
UPSTASH_REDIS_REST_TOKEN	Server secret
GEMINI_API_KEY	Server secret
NEXT_PUBLIC_MONAD_RPC_URL	Public
NEXT_PUBLIC_AGORA_AUSD_ADDRESS	Public
NEXT_PUBLIC_AGORA_FAUCET_ADDRESS	Public
NEXT_PUBLIC_MESH_TAB_LEDGER_ADDRESS	Public
NEXT_PUBLIC_PRIVY_APP_ID	Public

Server-side Privy verification and the paymaster policies are set in the Privy dashboard and hosting configuration and are not listed in the developer's report.

13. DECISION LOG

Decision	Reason	Consequence
Submit on Monad Testnet only	Judges can test without real money	No real-value claims
Use the issuer's official testnet AUSD	Real token on the sponsor's rails. Its faucet allows 10,000 per claim	Token fixed in the ledger constructor
Keep netting off-chain	Netting does not need a blockchain and is faster in the app	The ledger cannot verify it
Permissionless, per-call ledger	No member can move another's funds	No group-atomic settlement
Batch approve and payments into one	Fewer confirmations and a shorter wait	Median fell from about 2.8 s to about 1.3 s

Decision	Reason	Consequence
operation		
Embedded smart accounts with a paymaster	No wallet setup and no gas for users	Dependence on Privy and Pimlico
Per-currency baseline	An honest comparison for multi-currency tabs	Single-currency tabs show no reduction
Cap tabs at 20 members	Keep netting responsive	Larger groups are out of scope
Judges land in the sample tab	They can test with no sign-in	Real settlement needs testnet mode
Integer cents, converted at the contract boundary	No rounding drift	Redis counters still use HINCRBYFLOAT

Appendix A: Ledger Interface

constructor(address _ausdToken)

function payNettedDebt(string calldata groupId, address creditor, uint256 amount) external

function ausdToken() external view returns (address)

event DebtSettled(string groupId, address indexed debtor, address indexed creditor, uint256 amount)

Appendix B: On-Chain Evidence

Item	Transaction	Notes
Faucet claim by a fresh wallet	0x22dc9d3ec72c7d6a70cf2ecd8b58ae8c13e9832b6526c61e9dcdd64099a82e04	Received 10,000 AUSD
Script-executed proof settlement	0xafd413d0ebd3e9da42b92fe61a3f737a3a437108cf367c159d6861712bc829f0	Ben pays Aisha $16.77, block 69400512. Run by a deployment script with two sequential calls, before the batched flow
Live-app sponsored faucet claim	0x7a84c8a2b5e284041d8e52db9a35e7dfd1efef7b28e67a0ff8bfcb521ad0153f	Block 69887311, gas sponsored
Live-app batched settlement	0xb3e1987d65f5832a89c934094a9616dfb7b12d5930ea2387e07a6df42cb658c1	Block 69887324, approval and payment in one operation

Read on the Monad Testnet explorer. These links are provided by the developer. A reviewer can open each hash.

Appendix C: Timing Runs (Batched, Single Payment)

Run	Device	Network	Tap to confirmed (ms)
1	iPhone 15 Pro, Safari	5G	1,380
2	Mac M2, Chrome	Wi-Fi	1,210
3	iPhone 15 Pro, Safari	5G	1,420
4	Mac M2, Chrome	Wi-Fi	1,190
5	iPhone 15 Pro, Safari	Wi-Fi	1,260
6	Mac M2, Chrome	Hotspot	1,540
7	iPhone 15 Pro, Safari	Wi-Fi	1,220
8	Mac M2, Chrome	Wi-Fi	1,290
9	iPhone 15 Pro, Safari	5G	1,410
10	Mac M2, Chrome	Wi-Fi	1,250

Appendix D: Code References (as reported)

Topic	Location
Batched settlement and confirmation	components/tab/settlement-drawer.tsx (lines 200 to 220, 261 to 290)
Passkey prompt	components/ui/passkey-prompt-modal.tsx
Per-currency baseline	components/NettingCanvas.tsx (lines 409 to 489)
Cash Out and Virtual Card (simulations)	components/tab/cash-out-drawer.tsx, components/tab/virtual-card-drawer.tsx
Ping All	components/tab/roster-carousel.tsx (56 to 72), app/api/tab/[id]/remind/route.ts (73 to 98)
Default sample mode	lib/user.ts (line 345)
Developer toggle and security copy	app/settings/page.tsx (985 to 1020, 932)
Capacity message	components/tab/manage-members.tsx (line 94)
Lua scripts and sync	lib/redis-scripts.ts, lib/sync.ts, lib/onchain.ts
Ledger and deployment config	contracts/contracts/MeshTabLedger.sol, contracts/hardhat.config.js
Property test	test-netting-properties.mjs
Landing copy	content/landing.ts